



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №4

«Проект каталог. База данных»

по дисциплине

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«05» мая 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ПЛАН РАБОТЫ	3
2. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И МОДЕЛЕЙ.....	4
3. РЕАЛИЗАЦИЯ АДМИНКИ, ПРЕДСТАВЛЕНИЙ И СТРАНИЦ	6
4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ	10
5. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ	12
6. ИСХОДНЫЙ КОД	13
6.1. Модели базы данных	13
6.2. Настройка админки	15
6.3. Представления	16
6.4. Маршрутизация	17
6.5. HTML-шаблоны	17
6.6. Тесты.....	20
ВЫВОДЫ	24

1. ЦЕЛЬ И ПЛАН РАБОТЫ

Цель: реализовать второй этап проекта каталога товаров на Django: создать модели базы данных для хранения аккумуляторов, настроить админку Django для ввода тестовых данных, переделать представления так, чтобы страницы брали товары из базы данных, а также написать автоматизированные тесты.

План:

1. Изучить требования к практической работе.
2. Использовать проект каталога аккумуляторов из предыдущей практической работы как основу.
3. Спроектировать модели для хранения аккумуляторов и типов аккумуляторов.
4. Настроить ограничения допустимых значений на уровне полей и базы данных.
5. Настроить связь один-ко-многим между типом аккумулятора и товаром.
6. Подключить модели к админке Django.
7. Переделать страницу списка товаров и страницу детального описания товара для работы с базой данных.
8. Добавить поиск и фильтрацию по каталогу.
9. Подготовить фикстуру с тестовыми данными.
10. Написать тесты для моделей, связей, свойств, менеджеров, ограничений БД и представлений.
11. Проверить работу проекта и оформить отчет.

2. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И МОДЕЛЕЙ

Тема работы: аккумулятор.

Для каталога были выделены две основные сущности:

1. BatteryType - тип аккумулятора.
2. Battery - конкретный товар каталога.

Модель BatteryType используется для поля типа «один из». Это отдельная таблица, в которой хранятся возможные типы аккумуляторов:

- автомобильный;
- Power bank;
- AA/AAA;
- ИБП.

Модель Battery хранит сведения о товаре. Для аккумулятора используются следующие поля:

- артикул sku - строка, уникальное значение;
- название name - строка;
- бренд brand - строка;
- тип battery_type - внешний ключ на BatteryType;
- емкость capacity_mah - положительное число;
- напряжение voltage_v - положительное десятичное число;
- пусковой ток starting_current_a - положительное число, необязательное поле;
- вес weight_grams - положительное число, необязательное поле;
- цена price - положительное десятичное число;
- остаток stock_quantity - неотрицательное число;
- гарантия warranty_months - число от 0 до 120;
- поддержка быстрой зарядки fast_charge - логическое поле;
- описание description - текстовое поле.

Связь между моделями настроена как один-ко-многим: один тип аккумулятора может относиться ко многим товарам. Для этого в модели Battery используется ForeignKey на BatteryType.

Удаление типа аккумулятора защищено параметром `on_delete=PROTECT`. Это нужно, чтобы нельзя было удалить тип, если к нему уже привязаны товары.

Для проверки корректности данных используются:

- валидаторы Django: `MinValueValidator`, `MaxValueValidator`, `RegexValidator`;
- ограничения уровня базы данных `CheckConstraint`;
- уникальное ограничение `unique=True` для артикула `sku`.

В модели Battery также реализованы вычисляемые свойства:

- `is_heavy` - аккумулятор считается тяжелым, если вес больше или равен 5000 г;
- `is_high_capacity` - аккумулятор считается аккумулятором большой емкости, если емкость больше или равна 20000 мАч.

Для удобной выборки товаров реализованы менеджеры:

- `by_type()` - фильтрация по типу аккумулятора;
- `in_stock()` - товары в наличии;
- `high_capacity()` - аккумуляторы большой емкости;
- `heavy()` - тяжелые аккумуляторы.

3. РЕАЛИЗАЦИЯ АДМИНКИ, ПРЕДСТАВЛЕНИЙ И СТРАНИЦ

В админке Django зарегистрированы модели BatteryType и Battery.

Для BatteryType настроены:

- отображение названия и slug;
- поиск по названию и описанию;
- автоматическое заполнение slug по названию.

Для Battery настроены:

- отображение артикула, названия, бренда, типа, емкости, напряжения, цены и остатка;
- фильтрация по типу, бренду и поддержке быстрой зарядки;
- поиск по артикулу, названию, бренду и описанию;
- редактирование цены и остатка прямо из списка;
- readonly-поля для дат создания и обновления;
- отображение вычисляемых признаков «Тяжелый» и «Большая емкость».

Страница списка товаров открывается по адресу:

/

На ней отображаются товары из базы данных. Для каждого аккумулятора выводятся:

- тип;
- название;
- бренд;
- артикул;
- емкость;
- напряжение;
- гарантия;
- цена;
- наличие на складе;

- ссылка на детальную страницу.

На странице списка реализован поиск по названию, бренду, артикулу и описанию. Также есть фильтрация по типу аккумулятора и наличию товара на складе.

Страница детального описания товара открывается по адресу:

/product/<int:pk>/

На ней отображаются подробные характеристики конкретного аккумулятора из базы данных:

- название;
- бренд;
- артикул;
- тип;
- цена;
- описание;
- складской остаток;
- емкость;
- напряжение;
- пусковой ток;
- вес;
- поддержка быстрой зарядки;
- признак большой емкости;
- признак тяжелого аккумулятора;
- гарантия.

Страница «О магазине» открывается по адресу:

/about/

Она содержит краткое описание магазина аккумуляторов.

Скриншоты страниц сохранены в папке:

ПЗ4/screenshots/

Файлы скриншотов:

- product_list.png - список товаров;
- product_detail.png - детальная страница товара.

Рисунок 1. Список товаров каталога аккумуляторов

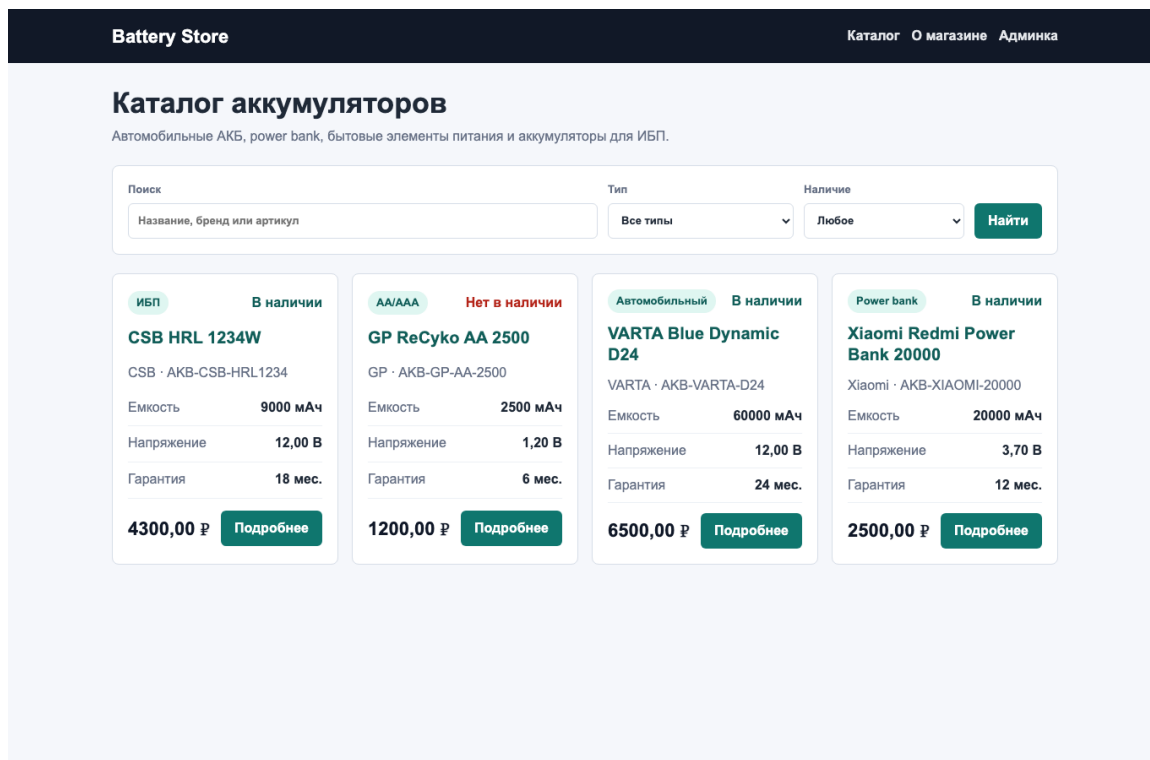


Рисунок 2. Детальная страница товара каталога аккумуляторов

[Назад к каталогу](#)

VARTA Blue Dynamic D24

VARTA · АКВ-VARTA-D24 · Автомобильный

6500,00 ₽

Автомобильный аккумулятор емкостью 60 Ач с пусковым током 540 А.

На складе: 4 шт.

Тип	Автомобильный
Емкость	60000 мАч
Напряжение	12,00 В
Пусковой ток	540 А
Вес	14500 г
Быстрая зарядка	Нет
Большая емкость	Да
Тяжелый	Да
Гарантия	24 мес.

4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Для проверки проекта использовался встроенный тестовый фреймворк Django на базе unittest.

Команда запуска тестов:

```
cd ПЗ4/catalog_project  
python3 manage.py test
```

Результат выполнения:

```
Found 21 test(s).  
System check identified no issues (0 silenced).  
Ran 21 tests  
OK
```

Были реализованы разные типы тестов:

1. Тест валидации положительной цены.
2. Тест валидации положительной емкости.
3. Тест валидации веса: значение может быть пустым, но не может быть нулевым.
4. Тест уникальности артикула.
5. Тест формата артикула.
6. Тест обратной связи от типа аккумулятора к товарам.
7. Тест on_delete=PROTECT для типа аккумулятора.
8. Тест вычисляемого свойства is_heavy.
9. Тест вычисляемого свойства is_high_capacity.
10. Тест менеджера by_type().
11. Тест менеджера in_stock().
12. Тест менеджера heavy().
13. Тест ограничения БД на положительную цену.
14. Тест ограничения БД на положительную емкость.
15. Тест уникального ограничения БД для артикула.
16. Тест доступности страницы списка товаров.

17. Тест вывода товаров из базы данных на странице списка.
18. Тест вывода товара из базы данных на детальной странице.
19. Тест ответа 404 для несуществующего товара.
20. Тест работы поиска.
21. Тест работы фильтрации по типу.

В тестах используются `setUpTestData` и `setUp`:

- `setUpTestData` применяется для создания общих объектов, которые можно использовать во всех тестах класса;
- `setUp` применяется для подготовки данных перед каждым отдельным тестом.

Также используется фикстура `catalog/fixtures/battery_catalog.json` с тестовыми типами аккумуляторов и товарами.

5. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ

- [x] Созданы необходимые модели для хранения товаров.
- [x] Для поля «тип аккумулятора» создана отдельная таблица BatteryType.
- [x] Настроена связь один-ко-многим между BatteryType и Battery.
- [x] Для связи используется on_delete=PROTECT.
- [x] Для полей прописаны ограничения допустимых значений.
- [x] Добавлены ограничения уровня базы данных CheckConstraint.
- [x] Настроена админка Django для ввода тестовых данных.
- [x] Страница списка товаров берет данные из базы данных.
- [x] Детальная страница товара берет данные из базы данных.
- [x] В каталоге есть поиск.
- [x] В каталоге есть фильтрация.
- [x] Добавление, удаление и редактирование товара доступно через админку.
- [x] Подготовлена фикстура с тестовыми данными.
- [x] Написаны тесты разных типов.
- [x] Количество тестов больше 10: реализован 21 тест.
- [x] Сделаны скриншоты страниц.

6. ИСХОДНЫЙ КОД

6.1. Модели базы данных

Листинг 1. Файл catalog/models.py

```
from decimal import Decimal

from django.core.validators import MaxValueValidator, MinValueValidator, RegexValidator
from django.db import models
from django.db.models import Q
from django.urls import reverse

class BatteryType(models.Model):
    name = models.CharField('Название типа', max_length=80, unique=True)
    slug = models.SlugField('Код для фильтра', max_length=90, unique=True)
    description = models.TextField('Описание', blank=True)

    class Meta:
        ordering = ('name',)
        verbose_name = 'Тип аккумулятора'
        verbose_name_plural = 'Типы аккумуляторов'

    def __str__(self):
        return self.name

class BatteryQuerySet(models.QuerySet):
    def by_type(self, battery_type):
        if isinstance(battery_type, BatteryType):
            return self.filter(battery_type=battery_type)
        return self.filter(battery_type__slug=battery_type)

    def in_stock(self):
        return self.filter(stock_quantity__gt=0)

    def high_capacity(self):
        return self.filter(capacity_mah__gte=20000)

    def heavy(self):
        return self.filter(weight_grams__gte=5000)

class Battery(models.Model):
    sku = models.CharField(
        'Артикул',
        max_length=32,
        unique=True,
        validators=[
            RegexValidator(
                regex=r'^[A-Z0-9-]+$ ',
                message='Артикул должен состоять из латинских заглавных букв, цифр и дефисов.',
            )
        ],
    )
    name = models.CharField('Название', max_length=120)
    brand = models.CharField('Бренд', max_length=80)
    battery_type = models.ForeignKey(
```

```

        BatteryType,
        on_delete=models.PROTECT,
        related_name='batteries',
        verbose_name='Тип аккумулятора',
    )
    capacity_mah = models.PositiveIntegerField(
        'Емкость, мАч',
        validators=[MinValueValidator(1), MaxValueValidator(500000)],
    )
    voltage_v = models.DecimalField(
        'Напряжение, В',
        max_digits=5,
        decimal_places=2,
        validators=[MinValueValidator(Decimal('0.01')),
MaxValueValidator(Decimal('100.00'))],
    )
    starting_current_a = models.PositiveIntegerField(
        'Пусковой ток, А',
        null=True,
        blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(3000)],
    )
    weight_grams = models.PositiveIntegerField(
        'Вес, г',
        null=True,
        blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(100000)],
    )
    price = models.DecimalField(
        'Цена, руб.',
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(Decimal('0.01'))],
    )
    stock_quantity = models.PositiveIntegerField(
        'Остаток на складе',
        default=0,
        validators=[MinValueValidator(0), MaxValueValidator(100000)],
    )
    warranty_months = models.PositiveSmallIntegerField(
        'Гарантия, мес.',
        default=12,
        validators=[MinValueValidator(0), MaxValueValidator(120)],
    )
    fast_charge = models.BooleanField('Поддержка быстрой зарядки', default=False)
    description = models.TextField('Описание', blank=True)
    created_at = models.DateTimeField('Создано', auto_now_add=True)
    updated_at = models.DateTimeField('Обновлено', auto_now=True)

    objects = BatteryQuerySet.as_manager()

    class Meta:
        ordering = ('name',)
        verbose_name = 'Аккумулятор'
        verbose_name_plural = 'Аккумуляторы'
        constraints = [
            models.CheckConstraint(condition=Q(price__gt=0),
name='battery_price_positive'),
            models.CheckConstraint(condition=Q(capacity_mah__gt=0),
name='battery_capacity_positive'),
            models.CheckConstraint(condition=Q(voltage_v__gt=0),
name='battery_voltage_positive'),
            models.CheckConstraint(condition=Q(stock_quantity__gte=0),
name='battery_stock_non_negative'),
            models.CheckConstraint(
                condition=Q(weight_grams__isnull=True) | Q(weight_grams__gt=0),
                name='battery_weight_null_or_positive',
            ),
            models.CheckConstraint(
                condition=Q(starting_current_a__isnull=True)
Q(starting_current_a__gt=0),

```

```

        name='battery_starting_current_null_or_positive',
    ),
    models.CheckConstraint(
        condition=Q(warranty_months__gte=0) & Q(warranty_months__lte=120),
        name='battery_warranty_valid_range',
    ),
]

@property
def is_heavy(self):
    return self.weight_grams is not None and self.weight_grams >= 5000

@property
def is_high_capacity(self):
    return self.capacity_mah >= 20000

def get_absolute_url(self):
    return reverse('product_detail', args=[self.pk])

def __str__(self):
    return f'{self.brand} {self.name} ({self.sku})'

```

6.2. Настройка админки

Листинг 2. Файл catalog/admin.py

```

from django.contrib import admin

from .models import Battery, BatteryType

@admin.register(BatteryType)
class BatteryTypeAdmin(admin.ModelAdmin):
    list_display = ('name', 'slug')
    search_fields = ('name', 'description')
    prepopulated_fields = {'slug': ('name',)}

@admin.register(Battery)
class BatteryAdmin(admin.ModelAdmin):
    list_display = (
        'sku',
        'name',
        'brand',
        'battery_type',
        'capacity_mah',
        'voltage_v',
        'price',
        'stock_quantity',
        'heavy_marker',
        'high_capacity_marker',
    )
    list_display_links = ('sku', 'name')
    list_editable = ('price', 'stock_quantity')
    list_filter = ('battery_type', 'brand', 'fast_charge')
    search_fields = ('sku', 'name', 'brand', 'description')
    autocomplete_fields = ('battery_type',)
    readonly_fields = ('created_at', 'updated_at')
    fieldsets = (
        ('Основное', {'fields': ('sku', 'name', 'brand', 'battery_type',
            'description')}),
    )

```

```

        'Характеристики',
        {
            'fields': (
                'capacity_mah',
                'voltage_v',
                'starting_current_a',
                'weight_grams',
                'fast_charge',
            )
        },
    ),
    ('Продажи', {'fields': ('price', 'stock_quantity', 'warranty_months')}),
    ('Служебные поля', {'fields': ('created_at', 'updated_at')}),
)

@admin.display(boolean=True, description='Тяжелый')
def heavy_marker(self, obj):
    return obj.is_heavy

@admin.display(boolean=True, description='Большая емкость')
def high_capacity_marker(self, obj):
    return obj.is_high_capacity

```

6.3. Представления

Листинг 3. Файл catalog/views.py

```

from django.db.models import Q
from django.shortcuts import get_object_or_404, render

from .models import Battery, BatteryType

def product_list(request):
    products = Battery.objects.select_related('battery_type')
    query = request.GET.get('q', '').strip()
    selected_type = request.GET.get('type', '').strip()
    selected_availability = request.GET.get('availability', '').strip()

    if query:
        products = products.filter(
            Q(name__icontains=query)
            | Q(brand__icontains=query)
            | Q(sku__icontains=query)
            | Q(description__icontains=query)
        )

    if selected_type:
        products = products.by_type(selected_type)

    if selected_availability == 'in_stock':
        products = products.in_stock()
    elif selected_availability == 'out_of_stock':
        products = products.filter(stock_quantity=0)

    context = {
        'products': products,
        'battery_types': BatteryType.objects.all(),
        'query': query,
        'selected_type': selected_type,
        'selected_availability': selected_availability,
    }

```



```

return render(request, 'catalog/product_list.html', context)

def product_detail(request, pk):
    product = get_object_or_404(Battery.objects.select_related('battery_type'), pk=pk)
    return render(request, 'catalog/product_detail.html', {'product': product})

def about(request):
    return render(request, 'catalog/about.html')

```

6.4. Маршрутизация

Листинг 4. Файл catalog_project/urls.py

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('catalog.urls')),
]

```

Листинг 5. Файл catalog/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('product/<int:pk>/', views.product_detail, name='product_detail'),
    path('about/', views.about, name='about'),
]

```

6.5. HTML-шаблоны

Для отображения страниц используются шаблоны из папки catalog/templates/catalog/.

Общий шаблон base.html содержит HTML-каркас страницы, меню навигации и CSS-оформление каталога.

Листинг 6. Файл catalog/templates/catalog/product_list.html

```

{% extends 'catalog/base.html' %}

{% block title %}Каталог аккумуляторов{% endblock %}

```

```

{% block content %}
<section class="page-header">
    <div>
        <h1>Каталог аккумуляторов</h1>
        <p>Автомобильные АКБ, power bank, бытовые элементы питания и аккумуляторы для
ИБП.</p>
    </div>
</section>

<form class="filters" method="get">
    <label>
        Поиск
        <input type="search" name="q" value="{{ query }}" placeholder="Название, бренд
или артикул">
    </label>
    <label>
        Тип
        <select name="type">
            <option value="">Все типы</option>
            {% for battery_type in battery_types %}
                <option value="{{ battery_type.slug }}" {% if selected_type ==
battery_type.slug %} selected{% endif %}>
                    {{ battery_type.name }}
                </option>
            {% endfor %}
        </select>
    </label>
    <label>
        Наличие
        <select name="availability">
            <option value="">Любое</option>
            <option value="in_stock" {% if selected_availability == 'in_stock' %}
selected{% endif %}>В наличии</option>
            <option value="out_of_stock" {% if selected_availability == 'out_of_stock'
%} selected{% endif %}>Нет в наличии</option>
        </select>
    </label>
    <button class="button" type="submit">Найти</button>
</form>

{% if products %}
<section class="grid">
    {% for product in products %}
        <article class="product-card">
            <div class="product-card_top">
                <span class="badge">{{ product.battery_type.name }}</span>
                {% if product.stock_quantity > 0 %}
                    <span class="status">В наличии</span>
                {% else %}
                    <span class="status status--empty">Нет в наличии</span>
                {% endif %}
            </div>
            <h2><a href="{{ product.get_absolute_url }}">{{ product.name
}}</a></h2>
            <p>{{ product.brand }} · {{ product.sku }}</p>
            <dl class="specs">
                <div>
                    <dt>Емкость</dt>
                    <dd>{{ product.capacity_mah }} мАч</dd>
                </div>
                <div>
                    <dt>Напряжение</dt>
                    <dd>{{ product.voltage_v }} В</dd>
                </div>
                <div>
                    <dt>Гарантия</dt>
                    <dd>{{ product.warranty_months }} мес.</dd>
                </div>
            </dl>
            <div class="meta-row">

```

```

        <span class="price">{{ product.price }} Р</span>
        <a          class="button"          href="{{          product.get_absolute_url
}}">Подробнее</a>
    </div>
</article>
{% endfor %}
</section>
{% else %}
    <section class="empty-state">
        <p>По заданным условиям аккумуляторы не найдены.</p>
    </section>
{% endif %}
{% endblock %}

```

Листинг 7. Файл catalog/templates/catalog/product_detail.html

```

{% extends 'catalog/base.html' %}

{% block title %}{{ product.name }}{% endblock %}

{% block content %}
<a class="back-link" href="{% url 'product_list' %}">Назад к каталогу</a>

<article class="detail-panel">
    <header class="page-header">
        <div>
            <h1>{{ product.name }}</h1>
            <p>{{ product.brand }} · {{ product.sku }} · {{ product.battery_type.name
}}</p>
        </div>
        <span class="price">{{ product.price }} Р</span>
    </header>

    <section class="detail-grid">
        <div>
            <p>{{ product.description|default:'Описание пока не добавлено.' }}</p>
            {% if product.stock_quantity > 0 %}
                <p class="status">На складе: {{ product.stock_quantity }} шт.</p>
            {% else %}
                <p class="status status--empty">Сейчас нет в наличии</p>
            {% endif %}
        </div>

        <dl class="specs">
            <div>
                <dt>Тип</dt>
                <dd>{{ product.battery_type.name }}</dd>
            </div>
            <div>
                <dt>Емкость</dt>
                <dd>{{ product.capacity_mah }} мАч</dd>
            </div>
            <div>
                <dt>Напряжение</dt>
                <dd>{{ product.voltage_v }} В</dd>
            </div>
            <div>
                <dt>Пусковой ток</dt>
                <dd>{% if product.starting_current_a %}{{ product.starting_current_a }}
A{% else %}Не указан{% endif %}</dd>
            </div>
            <div>
                <dt>Вес</dt>
                <dd>{% if product.weight_grams %}{{ product.weight_grams }} г{% else
%}Не указан{% endif %}</dd>
            </div>
        </dl>
    </section>

```

```

        <dt>Быстрая зарядка</dt>
        <dd>{% if product.fast_charge %}Да{% else %}Нет{% endif %}</dd>
    </div>
    <div>
        <dt>Большая емкость</dt>
        <dd>{% if product.is_high_capacity %}Да{% else %}Нет{% endif %}</dd>
    </div>
    <div>
        <dt>Тяжелый</dt>
        <dd>{% if product.is_heavy %}Да{% else %}Нет{% endif %}</dd>
    </div>
    <div>
        <dt>Гарантия</dt>
        <dd>{{ product.warranty_months }} мес.</dd>
    </div>
</dl>
</section>
</article>
{% endblock %}

```

Листинг 8. Файл catalog/templates/catalog/about.html

```

{% extends 'catalog/base.html' %}

{% block title %}О магазине аккумуляторов{% endblock %}

{% block content %}
<section class="page-header">
    <div>
        <h1>О магазине</h1>
        <p>Каталог рассчитан на тестовое наполнение через админку Django и быстрый поиск по товарной базе.</p>
    </div>
</section>

<section class="detail-panel">
    <p>Магазин специализируется на аккумуляторах для автомобиля, портативной техники, бытовых устройств и резервного питания.</p>
    <p>Для каждого товара хранится тип, артикул, бренд, емкость, напряжение, цена, складской остаток, гарантия и дополнительные характеристики.</p>
</section>
{% endblock %}

```

6.6. Тесты

Листинг 9. Файл catalog/tests.py

```

from decimal import Decimal

from django.core.exceptions import ValidationError
from django.db import IntegrityError, transaction
from django.db.models import ProtectedError
from django.test import TestCase
from django.urls import reverse

from .models import Battery, BatteryType

class BatteryValidationTests(TestCase):
    @classmethod

```

```

def setUpTestData(cls):
    cls.battery_type = BatteryType.objects.create(
        name='Автомобильный',
        slug='car',
        description='Стартерные аккумуляторы для легковых автомобилей.',
    )

def setUp(self):
    self.valid_data = {
        'sku': 'AKB-TEST-001',
        'name': 'Test Battery 60Ah',
        'brand': 'TestBrand',
        'battery_type': self.battery_type,
        'capacity_mah': 60000,
        'voltage_v': Decimal('12.00'),
        'starting_current_a': 540,
        'weight_grams': 14500,
        'price': Decimal('6500.00'),
        'stock_quantity': 5,
        'warranty_months': 24,
        'fast_charge': False,
        'description': 'Тестовый аккумулятор.',
    }

def build_battery(self, **overrides):
    return Battery(**(self.valid_data | overrides))

def test_price_must_be_positive(self):
    battery = self.build_battery(price=Decimal('0.00'))
    with self.assertRaises(ValidationError) as context:
        battery.full_clean()
    self.assertIn('price', context.exception.message_dict)

def test_capacity_must_be_positive(self):
    battery = self.build_battery(capacity_mah=0)
    with self.assertRaises(ValidationError) as context:
        battery.full_clean()
    self.assertIn('capacity_mah', context.exception.message_dict)

def test_weight_can_be_empty_but_not_zero(self):
    battery_without_weight = self.build_battery(weight_grams=None)
    battery_without_weight.full_clean()
    battery_with_zero_weight = self.build_battery(weight_grams=0)
    with self.assertRaises(ValidationError) as context:
        battery_with_zero_weight.full_clean()
    self.assertIn('weight_grams', context.exception.message_dict)

def test_sku_must_be_unique(self):
    Battery.objects.create(**self.valid_data)
    duplicate = self.build_battery(name='Duplicate Battery')
    with self.assertRaises(ValidationError) as context:
        duplicate.full_clean()
    self.assertIn('sku', context.exception.message_dict)

def test_sku_allows_only_expected_characters(self):
    battery = self.build_battery(sku='акб-001')
    with self.assertRaises(ValidationError) as context:
        battery.full_clean()
    self.assertIn('sku', context.exception.message_dict)

class BatteryRelationTests(TestCase):
    fixtures = ['battery_catalog.json']

    def test_type_reverse_relation_returns_related_batteries(self):
        car_type = BatteryType.objects.get(slug='car')
        self.assertEqual(car_type.batteries.count(), 1)
        self.assertEqual(car_type.batteries.first().sku, 'AKB-VARTA-D24')

```

```

def test_type_delete_is_protected_when_products_exist(self):
    car_type = BatteryType.objects.get(slug='car')
    with self.assertRaises(ProtectedError):
        car_type.delete()

class BatteryPropertyTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.battery_type = BatteryType.objects.create(name='Power bank', slug='power-
bank')

    def make_battery(self, **overrides):
        data = {
            'sku': 'AKB-PROP-001',
            'name': 'Property Battery',
            'brand': 'PropBrand',
            'battery_type': self.battery_type,
            'capacity_mah': 20000,
            'voltage_v': Decimal('3.70'),
            'weight_grams': 450,
            'price': Decimal('2500.00'),
            'stock_quantity': 2,
        }
        return Battery(**(data | overrides))

    def test_is_heavy_property(self):
        light_battery = self.make_battery(weight_grams=4999)
        heavy_battery = self.make_battery(sku='AKB-PROP-002', weight_grams=5000)
        self.assertFalse(light_battery.is_heavy)
        self.assertTrue(heavy_battery.is_heavy)

    def test_is_high_capacity_property(self):
        regular_battery = self.make_battery(capacity_mah=19999)
        high_capacity_battery = self.make_battery(sku='AKB-PROP-003',
capacity_mah=20000)
        self.assertFalse(regular_battery.is_high_capacity)
        self.assertTrue(high_capacity_battery.is_high_capacity)

class BatteryManagerTests(TestCase):
    fixtures = ['battery_catalog.json']

    def test_by_type_manager_filters_by_slug(self):
        products = Battery.objects.by_type('power-bank')
        self.assertEqual(products.count(), 1)
        self.assertEqual(products.get().sku, 'AKB-XIAOMI-20000')

    def test_in_stock_manager_excludes_empty_stock(self):
        products = Battery.objects.in_stock()
        self.assertTrue(products.filter(sku='AKB-VARTA-D24').exists())
        self.assertFalse(products.filter(sku='AKB-GP-AA-2500').exists())

    def test_heavy_manager_returns_heavy_batteries(self):
        products = Battery.objects.heavy()
        self.assertTrue(products.filter(sku='AKB-VARTA-D24').exists())
        self.assertFalse(products.filter(sku='AKB-XIAOMI-20000').exists())

class BatteryConstraintTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.battery_type = BatteryType.objects.create(name='ИБП', slug='ups')

    def setUp(self):
        self.valid_data = {
            'sku': 'AKB-DB-001',
            'name': 'DB Constraint Battery',

```

```

        'brand': 'DBBrand',
        'battery_type': self.battery_type,
        'capacity_mah': 9000,
        'voltage_v': Decimal('12.00'),
        'weight_grams': 2600,
        'price': Decimal('4300.00'),
        'stock_quantity': 1,
        'warranty_months': 12,
    }

    def test_db_check_constraint_rejects_negative_price(self):
        with self.assertRaises(IntegrityError):
            with transaction.atomic():
                Battery.objects.create(**(self.valid_data | {'price': Decimal('-1.00')}))

    def test_db_check_constraint_rejects_zero_capacity(self):
        with self.assertRaises(IntegrityError):
            with transaction.atomic():
                Battery.objects.create(**(self.valid_data | {'sku': 'AKB-DB-002', 'capacity_mah': 0}))

    def test_db_unique_constraint_rejects_duplicate_sku(self):
        Battery.objects.create(**self.valid_data)
        with self.assertRaises(IntegrityError):
            with transaction.atomic():
                Battery.objects.create(**(self.valid_data | {'name': 'Duplicate DB Battery'}))

class CatalogViewTests(TestCase):
    fixtures = ['battery_catalog.json']

    def test_product_list_page_is_accessible(self):
        response = self.client.get(reverse('product_list'))
        self.assertEqual(response.status_code, 200)

    def test_product_list_reads_products_from_database(self):
        response = self.client.get(reverse('product_list'))
        self.assertContains(response, 'VARTA Blue Dynamic D24')
        self.assertContains(response, 'AKB-XIAOMI-20000')

    def test_product_detail_reads_product_from_database(self):
        product = Battery.objects.get(sku='AKB-VARTA-D24')
        response = self.client.get(reverse('product_detail', args=[product.pk]))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, product.description)
        self.assertContains(response, '540 A')

    def test_product_detail_returns_404_for_missing_product(self):
        response = self.client.get(reverse('product_detail', args=[999]))
        self.assertEqual(response.status_code, 404)

    def test_search_filters_products(self):
        response = self.client.get(reverse('product_list'), {'q': 'VARTA'})
        self.assertContains(response, 'VARTA Blue Dynamic D24')
        self.assertNotContains(response, 'Xiaomi Redmi Power Bank 20000')

    def test_type_filter_filters_products(self):
        response = self.client.get(reverse('product_list'), {'type': 'aa-aaa'})
        self.assertContains(response, 'GP ReCyko AA 2500')
        self.assertNotContains(response, 'VARTA Blue Dynamic D24')

```

ВЫВОДЫ

В результате выполнения практической работы был реализован второй этап проекта каталога аккумуляторов на Django.

В ходе работы были созданы модели для хранения типов аккумуляторов и товаров, настроена связь один-ко-многим, добавлены ограничения допустимых значений полей и ограничения уровня базы данных. Для тестового ввода и редактирования данных была настроена админка Django.

Представления были переделаны так, чтобы список товаров и детальная страница получали данные из базы данных, а не из статического списка. В каталоге добавлены поиск и фильтрация, что соответствует общим требованиям заказчика.

Для проверки корректности работы проекта написан набор автоматизированных тестов. Тесты покрывают валидацию полей, свойства модели, связь между моделями, менеджеры, ограничения базы данных и работу представлений. Всего выполнен 21 тест, все тесты прошли успешно.

Таким образом, цель практической работы достигнута.